

بسم الله الرحمن الرحيم

King Abdulaziz University  
Faculty of Computing and information Technology  
Computer Science Department



# CPCS-214 Computer Architecture and Organization

## 02 - Great Ideas in Computer Architecture

# Introduction

- In 1960s and 1970s, a primary constraint on computer performance was the size of computer's memory
- Programmers often followed a simple credo
  - minimize memory space to make programs fast

# Introduction

- In the last decade, advances in computer design and memory technology have greatly reduced importance of small memory size in most applications
  - other than those in embedded computing systems

# Performance

- Successful programmers have always been concerned about performance of their programs
  - Because getting results to the user quickly is critical in creating successful SW
- Programmers interested in performance need to understand issues that have replaced simple memory model
  - Parallel nature of processors
  - Hierarchical nature of memories

# Energy Efficiency

- Today's programmers need to worry about energy efficiency of their programs running either on PMD or in Cloud
  - Requires understanding what is below your code
- Programmers who seek to build competitive versions of SW
  - Needs to increase their knowledge of computer organization

# What You Can Learn

- By the time you complete this course, you will be able to answer the following questions:

# What You Can Learn

- How are programs written in a high-level language, such as C or Java, translated into language of HW?
- How does the HW execute resulting program?
- Comprehending these concepts forms basis of understanding aspects of both the HW and SW that affect program performance

# What You Can Learn

- What is the interface between the SW and the HW?
- How does SW instruct HW to perform needed functions?



# What You Can Learn

- What determines performance of a program, and how can a programmer improve performance?
  - Depends on original program, SW translation of that program into computer's language, and effectiveness of HW in executing program
- What techniques can be used by HW designers to improve performance?
  - Introduce basic concepts of modern computer design

# What You Can Learn

- What techniques can be used by HW designers to improve energy efficiency?
- What can programmer do to help or hinder energy efficiency?

# What You Can Learn

- What are reasons for and consequences of recent switch from sequential processing to parallel processing?
- Since first commercial computer in 1951, what great ideas did computer architects come up with that lay the foundation of modern computing?

# What You Can Learn

- Without understanding answers to these questions
  - improving performance of your program on modern computer will be a complex process of trial and error, rather than scientific procedure driven by insight and analysis

# Program Performance

- Depends on combination of
  - Effectiveness of algorithms used in program
  - SW used to create and translate program into machine instructions
  - Effectiveness of computer in executing those instructions, which may include input/output (I/O) operations

# HW and SW Affect Performance

| Hardware or software component                   | How this component affects performance                                                          |
|--------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Algorithm                                        | Determines both the number of source-level statements and the number of I/O operations executed |
| Programming language, compiler, and architecture | Determines the number of computer instructions for each source-level statement                  |
| Processor and memory system                      | Determines how fast instructions can be executed                                                |
| I/O system (hardware and operating system)       | Determines how fast I/O operations may be executed                                              |

# Great Ideas in Computer Architecture

- Eight great ideas that computer architects have been invented in the last 60 years of computer design
- So powerful they have lasted long after the first computer that used them
- Newer architects demonstrating their admiration by imitating their predecessors

# Great Ideas in Computer Architecture

1. Design for Moore's Law
2. Use abstraction to simplify design
3. Make the common case fast
4. Performance via parallelism
5. Performance via pipelining
6. Performance via prediction
7. Hierarchy of memories
8. Dependability via redundancy





# Moore's Law

- States: integrated circuit resources double every 18–24 months
- As computer designs can take years, resources per chip can easily double or quadruple between start and finish of project
- Computer architects must anticipate where technology will be when design finishes rather than design for where it starts



# Use Abstraction

- Techniques to be more productive
  - Otherwise design time would lengthen as dramatically as resources grew by Moore's law
- Represent the design at different levels or representations: lower-level details are hidden to offer a simpler model at higher levels



# Make Common Case Fast

- Enhance performance better than optimizing rare case
- Common case is often simpler than rare case
- Implies that you know what the common case is, which is only possible with careful experimentation and measurement



COMMON CASE FAST

# Performance via Parallelism

- Computer architects have offered designs that get more performance by performing operations in parallel



# Performance via Pipelining

- A particular pattern of parallelism is so prevalent in computer architecture that it merits its own name: **pipelining**



# Performance via Prediction

- In some cases it can be faster on average to guess and start working rather than wait until you know for sure,
- Assuming that mechanism to recover from a misprediction is not too expensive and your prediction is relatively accurate



PREDICTION

# Hierarchy of memories

- Programmers want memory to be fast, large, and cheap
- Conflicting demands
- Hierarchy of memories
- Fastest, smallest, and most expensive memory per bit at the top, and slowest, largest, and cheapest at the bottom



H I E R A R C H Y

# Dependability via Redundancy

- Computers not only need to be fast; need to be dependable
- Physical device can fail
  - Make system dependable
    - Including redundant components that can take over when a failure occurs
- Restoring redundancy



**DEPENDABILITY**